Exercise 1: Paper Reviews and Amdahl's Law

Michele Arrigoni Francesco Foresti Kasra Ghasemipoo

Group 15

1.1 Paper Reviews

Evolution of the Graphics Processing Unit (GPU)

W. J. Dally, S. W. Keckler, and D. B. Kirk. *IEEE Micro*, vol. 41, no. 6, pp. 42–51, 2021.

1. Primary contribution

This article provides a historical synthesis of how GPUs evolved from fixed-function graphics hardware into programmable platforms for graphics, high-performance computing, and deep learning. The authors connect major developments, including programmable shaders, CUDA-enabled computation, and Tensor Cores, to the requirements of these applications. By examining hardware alongside programming tools and libraries, they explain how architectural capabilities and a supporting software ecosystem jointly enabled the adoption of GPUs beyond graphics.

2. Key insight of the contribution

The key insight is that GPU progress is driven by feedback between applications and architecture: programmability attracts new workloads, whose requirements then motivate specialized hardware. Tensor Cores illustrate this process by accelerating matrix multiply-and-accumulate operations and amortizing instruction overhead across many arithmetic operations. Together with suitable reduced-precision formats and software support, such specialization explains why the inference-performance trend called Huang's law exceeds the gains attributable to transistor scaling alone.

3. Opinion and reaction

I found it interesting that the drive for better video games helped create the computing platform behind the modern AI boom. The article presents this progression logically and makes the industry's direction understandable without relying on excessively detailed technical explanations. I also appreciated its emphasis on software, which makes clear why computational power alone is not enough to make a platform useful.

A Bridging Model for Parallel Computation

Leslie G. Valiant. *Communications of the ACM*, vol. 33, no. 8, pp. 103–111, 1990.

1. Primary contribution

Valiant introduces the bulk-synchronous parallel (BSP) model as a common framework between parallel software and hardware, with a role similar to that of the von Neumann model in sequential computing. The model combines processing and memory components, point-to-point communication, and controlled synchronization in units called supersteps. The paper supports this proposal with results on implementing shared-memory features and parallel algorithms on BSP, and on implementing BSP efficiently using different communication networks.

2. Key insight of the contribution

The key insight is that a program should expose more parallel work than the physical processors can execute at once. This parallel slackness gives the compiler room to distribute work, pipeline communication, and absorb uneven memory-access loads. Together with synchronization at a suitable granularity, it allows many low-level details to be managed automatically with only a constant-factor efficiency loss, provided enough parallelism and communication capacity are available.

3. Opinion and reaction

I found it interesting that the paper treats a shared model for software and hardware as an essential part of making parallel computing practical. The idea of exposing more parallelism than the machine can execute at once seems counterintuitive at first, but its role in handling communication and uneven workloads makes the argument convincing. I would have liked a more detailed programming example, since the efficiency arguments are harder to connect to an implementation than the basic model itself.

1.2 Amdahl's Law

1. Derivation

Let T_1 be the sequential execution time of a fixed workload, P the fraction of this time that can be parallelized, and N the number of processors. The serial part takes $(1-P)T_1$. Assuming perfect load balancing, no parallel overhead, and an ideal N-fold acceleration of the parallel part, its execution time is PT_1/N. Hence

$$T_N = (1-P)T_1 + \frac{PT_1}{N} = T_1\left((1-P) + \frac{P}{N}\right). \tag{1}$$

The overall speedup is therefore

$$\boxed{S(N) = \frac{T_1}{T_N} = \frac{1}{(1-P) + P/N}}. \tag{2}$$

For $0 \le P < 1$,

$$\lim_{N\to\infty} S(N) = \frac{1}{1-P}. \tag{3}$$

The serial fraction sets the asymptotic limit. For the special case $P = 1$, the ideal speedup is $S(N) = N$ and there is no finite serial-fraction limit.

2. Correctness for real applications

Amdahl's law is correct under its assumptions. It is not a universal prediction for every parallel execution: using N as the acceleration of the parallel portion assumes that overhead and other architectural effects can be neglected.

Performance predicted too high. Communication, synchronization, thread creation, and load imbalance increase the actual time. Consider a computation with an ideal 100× speedup whose processors exchange intermediate results over a network. If communication becomes a bottleneck, the actual speedup is below 100×. Writing

$$T_N = T_1\left((1-P) + \frac{P}{N}\right) + T_{\text{overhead}}(N)$$

makes clear why positive overhead reduces the ideal speedup.

Performance predicted too low. A working set that exceeds the cache of one processor may fit into the aggregate caches when partitioned across several processors. Reducing slow memory accesses can accelerate the parallel part by more than N, so actual speedup can exceed the prediction of Equation (2), and can even be superlinear. This changes the assumed acceleration of the parallel portion; it does not invalidate the time-decomposition argument.

Changing the workload. In weather forecasting, more processors are often used to increase grid resolution rather than to finish an unchanged problem sooner. Amdahl's fixed-workload limit does not directly characterize this comparison. Weak scaling studies increasing the workload in proportion to the number of processors, while Gustafson's law addresses scaled speedup using the serial fraction of a parallel run. These perspectives change the workload assumptions used in the fixed-size Amdahl model.

3. Numerical calculations and comparison

(a) $P = 0.5$ **and** $N = 4$

$$S_A = \frac{1}{(1 - 0.5) + 0.5/4} = \frac{1}{0.5 + 0.125} = \boxed{1.6}.$$

The parallel execution takes $0.625T_1$, and its efficiency is $S_A/4 = 0.4 = 40\%$.

(b) $P = 0.2$ **and** $N = 8$

$$S_B = \frac{1}{(1 - 0.2) + 0.2/8} = \frac{1}{0.8 + 0.025} = \boxed{\frac{40}{33} \approx 1.212}.$$

The parallel execution takes $0.825T_1$, and its efficiency is $S_B/8 = 5/33 \approx 15.15\%$.

(c) Comparison

Case	P	N	Speedup	Limit as $N \to \infty$
A	0.5	4	1.6	2
B	0.2	8	1.212...	1.25

Case A is faster relative to its sequential baseline despite having fewer processors. Its larger parallelizable fraction leaves a smaller serial bottleneck. In case B, 80% of the original time remains serial, so even arbitrarily many processors cannot produce a speedup above 1.25.

These cases change both P and N, so their difference cannot be attributed to processor count alone. For any fixed $P > 0$, adding processors increases ideal speedup, but with diminishing returns when $P < 1$. Reducing the serial fraction is essential for effective scaling.

For comparison, keeping $P = 0.5$ and increasing only N from four to eight would give

$$S(8) = \frac{1}{0.5 + 0.5/8} = \frac{16}{9} \approx 1.778.$$

1.3 Willingness to Present

We are willing to present the following in English:

- Paper review 1: *Evolution of the Graphics Processing Unit (GPU).*
- Paper review 2: *A Bridging Model for Parallel Computation.*
- The exercise on Amdahl's law.